

The biomimetic pipeline — measured enamel → printable lattice

The problem it solves: the measured morphometrics and the parametric lattice computer-aided-design (CAD) model existed *separately* — the CAD was driven by hand-picked parameters. This pipeline makes the biology drive the geometry.

UPSTREAM • the measurement stack

microct_pipeline/ — upstream measurement repository

X-ray micro-computed-tomography (micro-CT) stack → 101 slices → `detect_rods.py` (watershed) → `track_rods_piv.py` (particle-image-velocimetry optical-flow tracking → three-dimensional rod tracks; pitch / yaw / tilt)
→ `compute_tortuosity.py` → `analysis_decussation.py` (Fourier band periodicity, k -means angle clustering) → `som_approach/` (self-organising-map band segmentation)

STAGE 0 • INGEST

ingest/ — six ingesters, adapter pattern; one per upstream tool

`hsb_ | piv_ | smoothed3d_ | som_ | tracking_ | validation_ingester` → `merge.py`
Each adapter knows *one* upstream format. Adding a new instrument = adding one ingester — nothing else changes.

`morphometrics.json` CONTRACT 1 —
`jsonschema.validate` on load (`merge.py`)

required: `schema_version` · `specimen_id` · `units` · `provenance` · `depth_profiles` · `angle_stats` · `bands`

STAGE 1 • MAPPING — biology becomes geometry

mapping/

pitch profile → **RING_ROTATION**: integrate signed pitch, $\phi(z) = \int p(z) dz$; classify against linear / accelerating / sigmoid, lowest root-mean-square error. If no shape fits within 10%: **TWIST_TYPE** = **measured** + full (z, ϕ) spline.

self-organising-map bands → **bridge geometry**: $N_{\text{bridge}} = \text{clip}(\text{round}(H/\lambda), 2, 8)$; **RING_ROTATION**[0] = band-0 direction; **CENTER_SPACING** = mean band width

rod morphology → **ROD_DIAMETER**, **ROD_TAPER_FACTOR**; >15% non-monotone → **per_ring_diameter** + *radially-graded* generator

tortuosity → **twist_perturbation_amplitude_deg** (clipped $[0, 15]^\circ$; drives the hierarchical-twist generator)

SCALE + CLAMP: real rod $\approx 4 \mu\text{m}$; printed rod = 2 mm ⇒ **biology_scale_factor** = $500\times$. Every feature clamped to **sla_min_feature_mm** = 0.3 mm — and every clamp is logged with its raw AND clamped value into **provenance.scale_clamps**. The mapping is auditable.

4 LATTICE INVARIANTS enforced HERE — bridge/rod ratio, lattice density, junction-sphere plate clearance, taper $\in [0, 1]$ — so no downstream stage ever fails validation.

`cad_parameters.json` CONTRACT 2 —
`jsonschema.validate` on load (`feature_to_cad`)

stock CAD keys verbatim (**ROD_DIAMETER**, **CENTER_SPACING**,
RING_ROTATION, **TWIST_TYPE**, ...) + new-model extensions

STAGE 2 • CAD + MESH

generators/ — runs in *cad_env*, then *sfepy_env*

`cad_runner.py` → `stock lattice_cad.py` (CadQuery) 2a. `cad_integrity.py` ← integrity check *post-CAD, pre-mesh* `mesh_runner.py` → `stock lattice_mesh.py` (Gmsh)

NON-INVASIVE: nothing inside `cad_modeling/` is ever modified. Stock tools are invoked through **conda run** with their existing command-line contracts intact. **Two separate conda environments because the dependency stacks genuinely conflict.**

STAGE 3 • FINITE-ELEMENT ANALYSIS (FEA) — the strain solver

fea/strain_solver.py

The stock solver prescribes a **DISPLACEMENT**. This pipeline does not. It solves for the displacement that produces a target stress: seed d_0 → run FEA → read average von-Mises stress → if $|VM - 200|/200 \leq \text{tol}$, accept; else $d \leftarrow d \cdot (200/VM)$ and re-run. The mesh is held fixed — only the displacement changes.

WHY A TARGET STRESS: every design is then evaluated at the same load state ($\approx 200 \text{ MPa}$). It decouples objective quality from incidental stiffness differences — otherwise a stiffer lattice would “win” for reasons that have nothing to do with the objective. Records **critical_strain_at_200MPa** as a first-class metric.

STAGES 4–5 • METRICS → PLUGGABLE OBJECTIVE

metrics/ · *objectives/registry.py*

crack deflection: rasterise per-element principal stress onto a 32^3 grid → gradient field → seed **64 streamlines** on the bottom face → integrate to the top → report mean / 90th-percentile / max of arc-length ÷ chord-length. **biomimicry score**: z -scored Euclidean distance across five measured-vs-generated feature pairs. Smaller = closer to the real biology.

Every configuration file under `config/objectives/` becomes a loadable objective with `.score(metrics)` and a direction. Five ship: **crack_deflection** (default) · **toughness** · **stiffness_target** · **biomimicry_score** · **composite**. A **stress_target** block drives the strain solver, so every objective is load-normalised by construction.

STAGE 6 • ORCHESTRATION — four modes, one core

orchestration/

`pipeline.py` one run | `sweep.py` parametric sweep | `optimize.py` Optuna Bayesian optimisation | `closed_loop.py` closed-loop inversion

CLOSED LOOP: run N trials, take the top- K by objective, then **INVERT the mapping** back into morphometric space and write

`biomimicry_targets.json` — the rod diameters, band widths, wavelengths and directions to go look for in a real specimen. The best simulated designs become measurement targets for the next specimen.